TP11 : Plus court chemin par l'algorithme de Dijkstra

CPES « Sciences des données, arts et cultures » : Introduction à l'algorithmique

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

06 mai 2024

Information importante : Un devoir sur table d'une heure aura lieu le 14 mai 2024 (dans une semaine), de 8h à 9h. Il sera suivi par la fin de la correction de ce TP de 9h à 10h. Les modalités du devoir sont les mêmes que lors des précédents : pas d'appareil numérique et 10 feuilles recto-verso de contenu maximum pendant l'épreuve.

Dans ce TP, nous allons implémenter l'algorithme de Dijsktra, afin de déterminer tous les plus courts chemins entre un sommet choisi par avance et tous les autres sommets qui sont accessibles.

Structure de graphe et file prioritaire

L'algorithme de Dijkstra se base sur deux structures de données que nous avons déjà implémentées au moins partiellement : un graphe non-orienté et pondéré sur lequel calculer les plus courts chemins, et une file de priorité qui doit nous permettre d'ordonner les sommets du graphe par distance décroissante au sommet de départ.

Question 1 Pour implémenter le graphe non-orienté et pondéré, recopier le code de la classe *Graphe* et de la classe *Sommet* écrit pour le DM3 (vous pouvez utiliser la correction pour ce faire). N'oubliez pas de recopier la fonction `ajout_arete_ponderee`.

Question 2 Comme vu en cours, il existe plusieurs façons d'implémenter une file de priorité. Nous allons exploiter l'implémentation du Tas-Min écrite au TP10 pour implémenter une file prioritaire ayant en tête le plus petit élément. Recopier le code du TP10 relatif au Tas-Min, incluant donc les fonctions : `gauche`, `droite`, `construction_tas_min` et `entasser_min`.

Question 3 Il manque à ce stade deux fonctions pour pouvoir utiliser un Tas-Min en tant que file prioritaire : l'extraction du minimum (et son retrait) ainsi que l'ajout d'un élément. Comme l'ajout n'est pas utile pour l'algoritme de Dijkstra, nous allons nous concentrer sur l'extraction du minimum et son retrait. Écrire une fonction `extraire_min` prenant en paramètre une liste Python `A` représentant un Tas-Min. Cette méthode devra **retourner un tuple** dont l'élément de gauche est le minimum extrait, et la

valeur de droite le Tas-Min ne contenu plus l'élément extrait. Pour ce faire, vous pouvez implémenter le pseudo-code de l'algorithme 5 du cours « Tas, Files de priorité et algo de Primm » (dans le PDF `slides10-Tas_File_de_Priorite_Primm.pdf` du dossier Cours du Moodle). *Attention : Comme déjà vu plusieurs fois, les pseudo-codes sont généralement écrits en indiçant les listes à 1, alors qu'en Python, elles commencent à 0... Il faut donc légèrement adapter le code en ce sens.*

Algorithme de Dijkstra

Dans l'algorithme de Dijkstra, deux informations sont mises à jour sur les sommets : la *distance* connue à l'instant t entre le sommet de départ et le sommet courant, et le *parent*, à savoir le sommet précédent à partir duquel on a pu calculer la *distance*.

Question 4 Écrire le code d'une nouvelle classe *SommetDijkstra*, héritante de la classe *Sommet*, contenant un constructeur. Ce dernier prendra en paramètres, en plus de `self`, `id` et `label` ; les deux paramètres usuels pour les sommets. Le constructeur fera appel au constructeur de la classe *Sommet*, et initialisera deux attributs `distance` et `parent`, avec les valeurs (respectives) `math.inf` (penser à importer la librairie math) et `None`.

Question 5 Ajouter dans votre code la fonction `remonter_valeur`, dans le fichier `remonter_valeur.py` du dossier TP-TP/11 dans le Moodle. Cette fonction permet, dans l'algorithme de Dijkstra, de remonter un sommet dans la file de priorité lorsque sa valeur de distance est modifiée : sinon, le Tas-Min n'en est plus un et on ne remonte plus le sommet avec la plus petite distance pour les prochaines itérations. **Un appel à cette fonction est à insérer dans le code de la question 7, entre les lignes 11 et 12 du pseudo-code de l'algorithme 1 mentionné dans ladite question.**

Question 6 Ajouter à la classe *SommetDijkstra*, la méthode suivante :

```
def __lt__(self, other):
    return self.distance < other.distance
```

Celle-ci permet de re-définir l'opérateur < pour la classe *SommetDijkstra*. En d'autres termes, si on a deux objets *SommetDijkstra* A et B, alors écrire `A < B` revient à écrire `A.distance < B.distance`. Cela est nécessaire pour construire une file de priorité sur des objets *SommetDijkstra*.

Question 7 En partant du pseudo-code de l'algorithme 1 fourni à la slide 20 du cours sur les plus courts chemins, écrire une fonction `dijkstra` prenant en paramètre `graphe` un objet de la classe *Graphe*, ici un graphe non-orienté et pondéré ; ainsi qu'un ID de sommet `s`, l'ID du sommet à partir duquel calculer les plus courts chemins. *Indications : Commencez votre code par l'initialisation de* `S` *en tant qu'ensemble Python et* `Q` *en tant que liste Python. Ajoutez à* `Q` *les sommets SommetDijkstra, puis ensuite appliquez* `construction_tas_min` *à* `Q`.

Question 8 Reproduire le graphe de la Figure 2 du DM3, et, en utilisant l'algorithme implémenté, donner le plus court chemin enter le sommet 7 et et le sommet 11. Vérifier à la main que le résultat obtenu est bien le plus court chemin attendu.